



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 06

Linked Lists





THIS WEEK'S AGENDA

- 01. Issues with Arrays
- 02. Introduction to Linked Lists
- 03. The Basic Structure
- 04. Types of Linked Lists
- 05. The Structure of a Node
- 06. The Linked List Class
- 07. Algorithms of Linked Lists
- 08. Singly Linked Lists





ISSUES WITH ARRAYS





ISSUES WITH ARRAYS

- Recall that arrays have the following properties:
 - They are a collection of a number of values of similar data types.
 - They require contiguous memory locations.
 - Elements can be accessed directly (array[i]). ***Think of the time complexity.***
 - Searching requires us to traverse the whole array of size N. ***Again, think of the time complexity.***





ISSUES WITH ARRAYS

- Insertion and Deletion of a value at a location i are cumbersome:
 - Insertion requires us to create a new array of size $(N+1)$ in which we copy all values of the original array at location i one place to the right.
 - Deletion requires us to move all items of the original array at location i by one place to the left.
 - Large array allocations are impossible if memory is scarce.





ISSUES WITH ARRAYS

- Insertion of value 10 at index 2
 - Move all values from index 2 to index 5, one place to the right

Value	1	4	5	6	3	2
index	0	1	2	3	4	5

1	4	5	6	3	2	
0	1	2	3	4	5	6

10

1	4	10	5	6	3	2
0	1	2	3	4	5	6

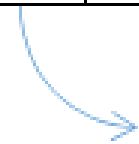




ISSUES WITH ARRAYS

- Deletion of value 4 from index 1
 - Move all values from index 2 to 5 one place to the left

1	4	5	6	3	2
0	1	2	3	4	5



1		5	6	3	2
0	1	2	3	4	5



4

1	5	6	3	2
0	1	2	3	4





INTRODUCTION TO LINKED LIST





INTRODUCTION TO LINKED LISTS

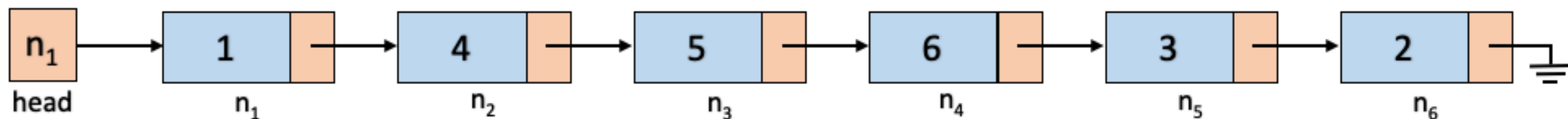
- Instead of creating a contiguous allocation of memory, we create nodes dynamically on demand and link them through pointers.
- There is no limit on the number of nodes that you can create (memory is the only limit).
- Insertion and Deletion become easier in the case of linked lists because we now just need to update a few pointers.
- A **linked list** can be thought of as a linked data structure - which has a collection of components, called **nodes**.
- Every node (except the last node) contains the address of the next node.





BASIC STRUCTURE OF LINKED LISTS

- Thus, every node in a linked list has two components:
 - (data) to store the relevant information,
 - (node) to store the address, called the **link**, of the next node in the list.
- The same data that we were previously storing in an array, may also be stored in a linked list which would look something like this:



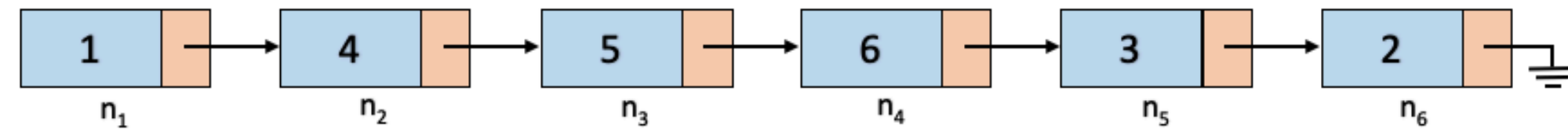


TYPES OF LINKED LISTS

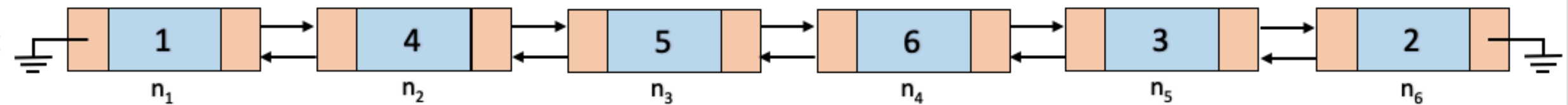


TYPES OF LINKED LISTS

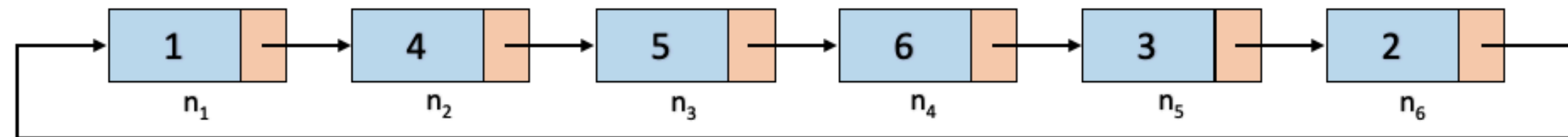
Singly LinkedList



Doubly LinkedList



Circular LinkedList





THE STRUCTURE OF A NODE





THE STRUCTURE OF A NODE

- At the foundation of every linked data structure lies the Node class.
- The Node class defines what a node is, as shown in the previous slide - data and a link (in the form of a pointer).

```
3  class Node {  
4      public:  
5          int value;  
6          Node* next;  
7  
8          Node() : value(0), next(nullptr) {}  
9          Node(int val) : value(val), next(nullptr) {}  
10 };
```





THE → NOTATION

- In C++, the → symbol is called the arrow operator and is primarily used to access members (attributes/data or methods) of an object or a struct through a pointer.
- When you have a pointer to an object or a struct, instead of dereferencing the pointer and then accessing the member variable or function using the dot (.) operator, you can instead use the → operator.
- Lets say we have the Node class with the member variable called value, as shown in the previous slide.





THE → NOTATION

- Then, if you have a pointer to a Node object, you can access its members as follows:
 - `Node *node1 = new Node;`
 - `node1->value = 15;`
 - `std::cout << "Node1 value: " << node1->value << std::endl;`
- Without using the arrow operator, this would be:
 - `(*node1).value = 15;`





LINKED LIST - THE CLASS





LINKED LIST - THE CLASS

```
12  class LinkedList {  
13      public:  
14          Node *head; // head of the list  
15          Node *tail; // tail of the list  
16      private:  
17          // Constructor to initialize an empty list  
18          LinkedList() : head(nullptr), tail(nullptr) {}  
19  };
```





```

void printList() {
    // Starting from the head of the list
    Node *current = head;
    if (current == nullptr) {
        std::cout << "List is empty." << std::endl;
        return;
    }
    while (current != nullptr) {
        std::cout << current->value << " -> ";
        current = current->next;
    }
    std::cout << "NULL" << std::endl;
}
  
```





ALGORITHMS OF LINKED LIST





LINKED LIST - ADD TO FRONT METHOD

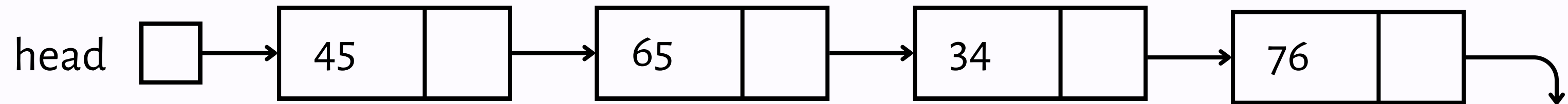
```
void addToFront(int value) {  
    Node *newNode = new Node(value);  
    if (head == nullptr) {  
        head = newNode;  
        tail = newNode;  
    } else {  
        newNode->next = head;  
        head = newNode;  
    }  
}
```





LINKED LIST - ADD TO FRONT METHOD

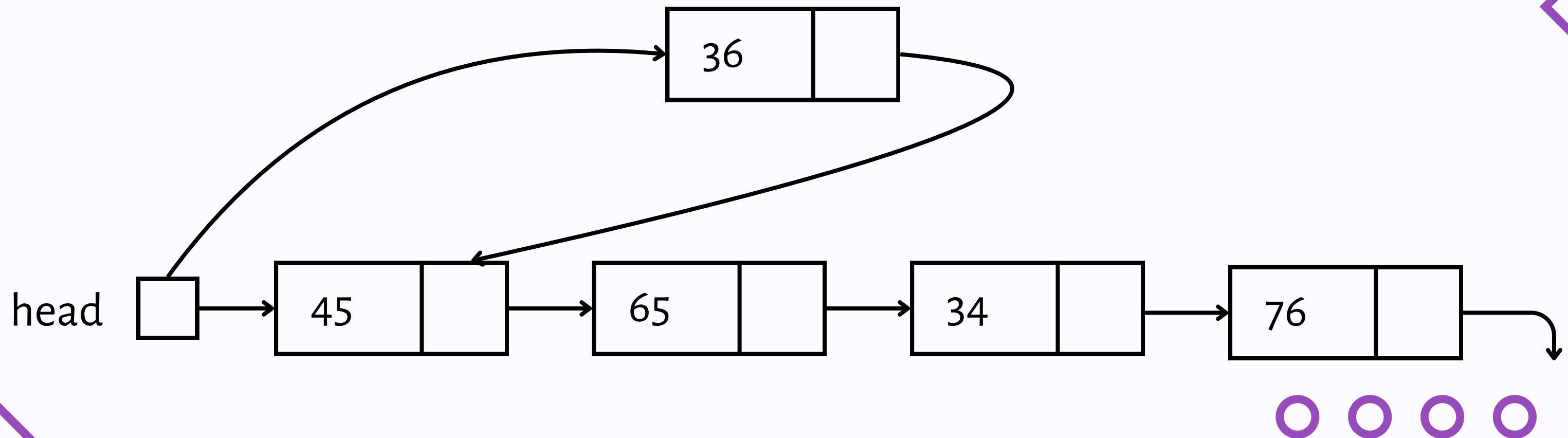
- The current state of the linked list is as shown below:





LINKED LIST - ADD TO FRONT METHOD

- The state of the linked list after adding a new node (with data 36) is as shown below:





LINKED LIST - ADD TO REAR METHOD

- Think of how you can write the AddToRear(int value) function.
- Make sure that you think of all possible cases, for example:
 - You will have to think about the case when the list is empty.
 - If it is not empty, would you just need to adjust the tail pointer?
 - If it is not empty, would you just need to adjust the head pointer?
- Are there any other possible edge/special cases?





SUMMARY





SUMMARY

- This session was an introduction to the linked list data structure.
- The linked list data structure is a linked data structure based on the idea of nodes - data elements with a value and a link.
- Linked list typically support the add, delete, find and remove functions at the two ends of the list.
- Other linked data structures can also be made such as stacks, queues, and double ended queues (also called dequeues).





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 06





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

